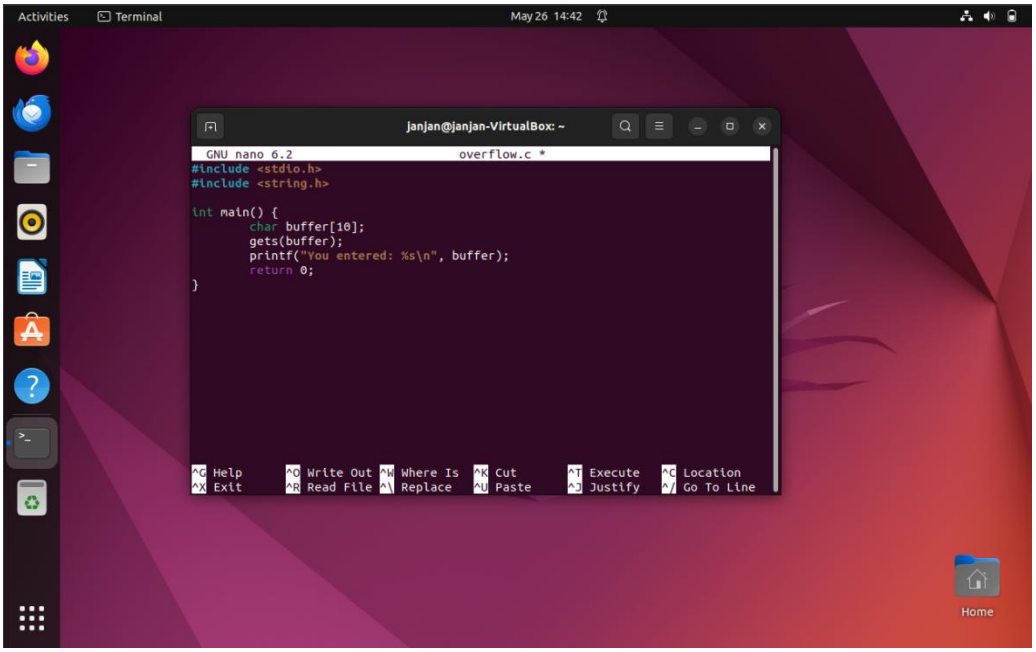
Jan Harvey C. Javier **05-26-2025**
BSIT-3B **Sir Benedick Sarmiento**

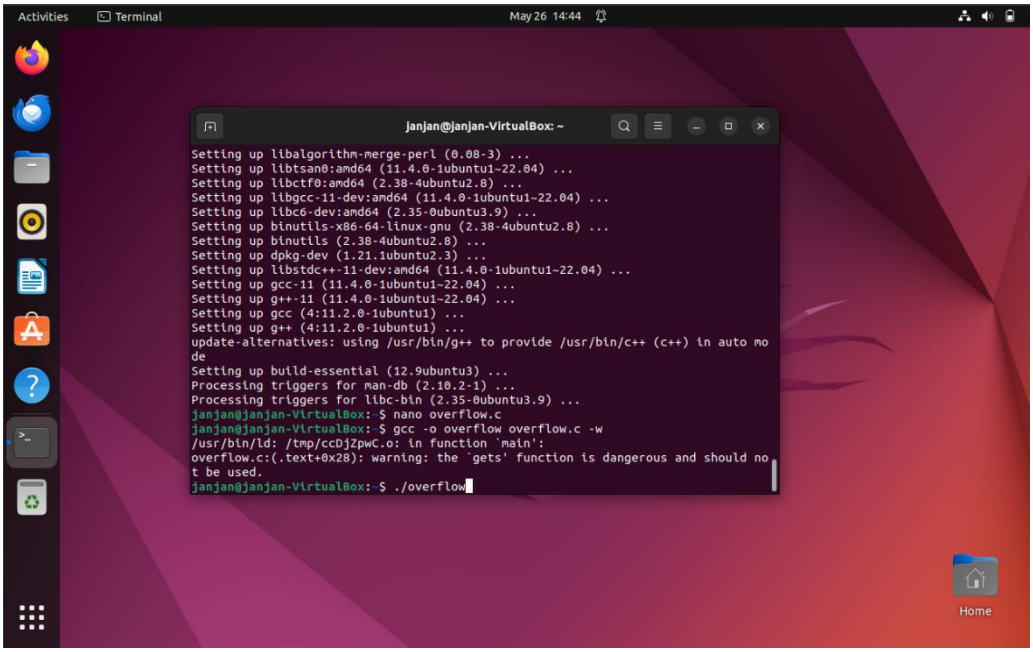
Common Security Vulnerabilities and Attacks

1. Buffer Overflow (System-level)

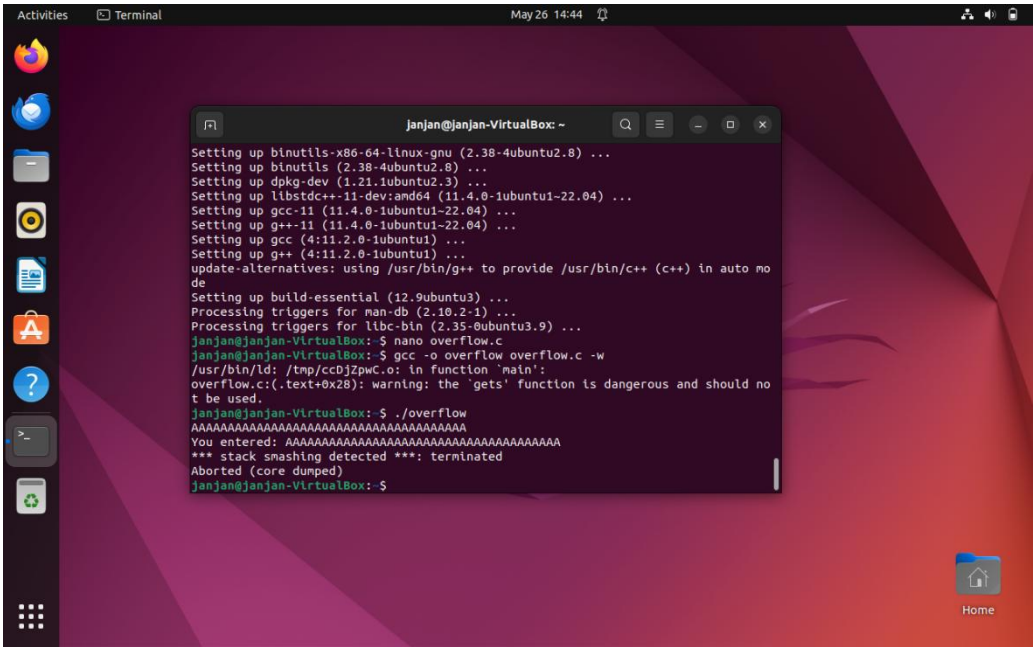
- Writing the code



- Executing the code



- System buffer overflows



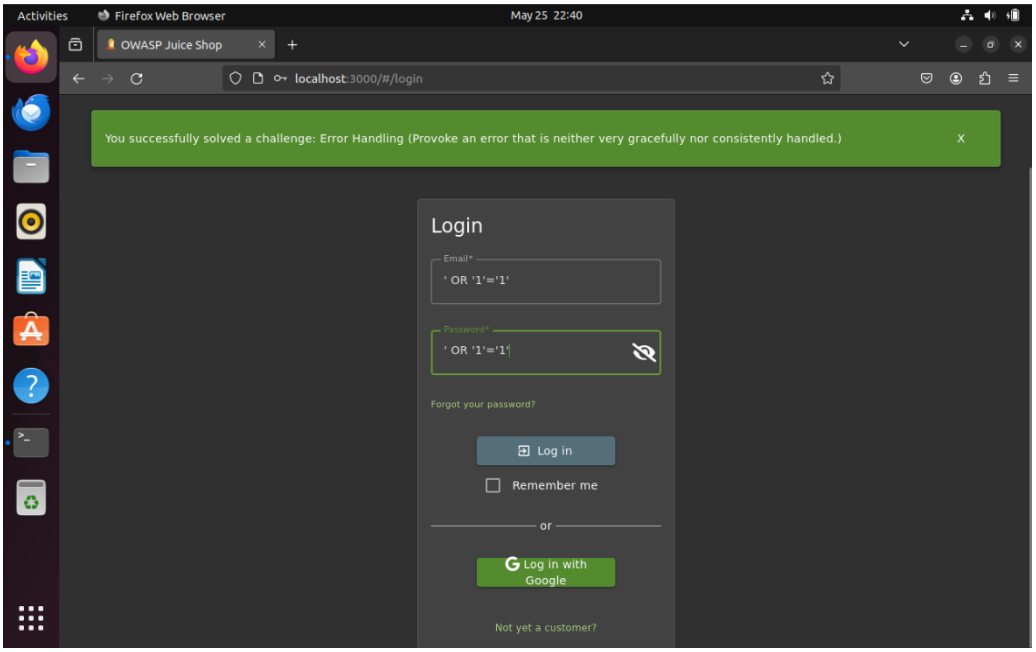
Summary: A buffer overflow happens when a program writes more data into a memory buffer than it can hold, overwriting adjacent memory. Luckily, after executing the code, my ubuntu operating system in virtual machine didn't crash.

Question: How could modern compilers mitigate this?

Answer: Modern compilers mitigate this vulnerability by removing the unsafe function get(). This function is removed in later versions. Compilers like C has stack canaries which is a has a special value placed on the stack to detect overflows. Additionally, modern OS' marks this stack as non-executable, preventing injected code from running if a buffer overflows.

2. SQL INJECTION

- Injecting the SQL in login form (OWASP Juice Shop)



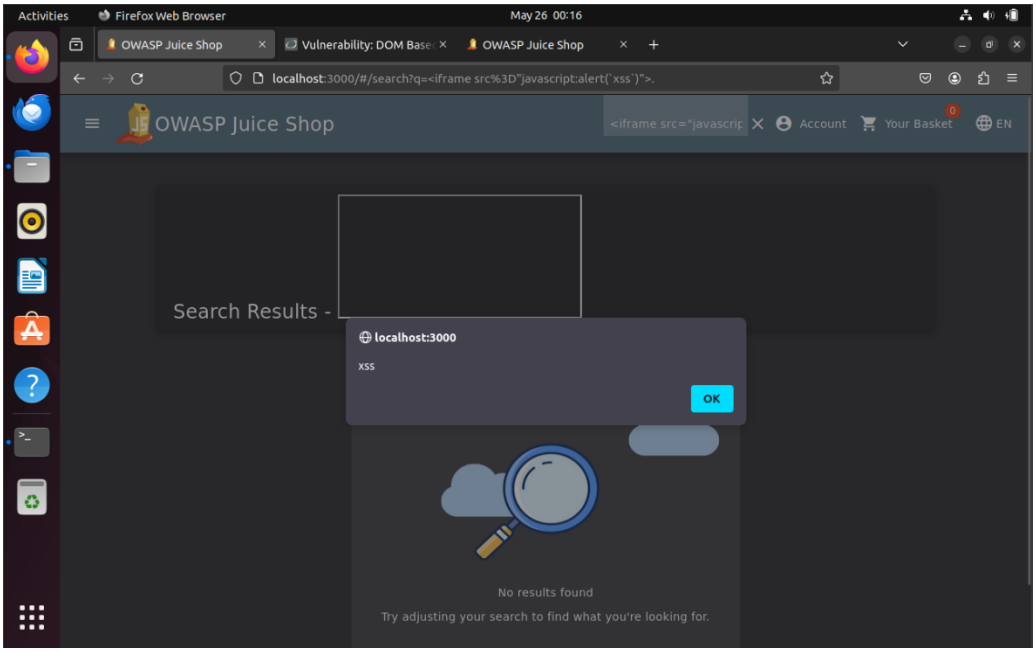
Summary: After injecting the ‘ OR ‘1’ = ‘1’ I was able to login in the OWASP Juice Shop and even got an achievement for doing so. Because of the SQL I injected I was able to login without being prevented. This happens when a user input is not properly filtered, allowing me (the attacker) to insert and modify SQL queries.

Mitigation: Identify how parameterized queries prevent this.

Answer: Parameterized queries prevent SQL injection by separating SQL code from user-supplied data. This separation ensures that user input is always treated as data, not executable SQL code.

3. Cross-Site Scripting (XSS)

- Executing alert script in OWASP Juice Shop search box



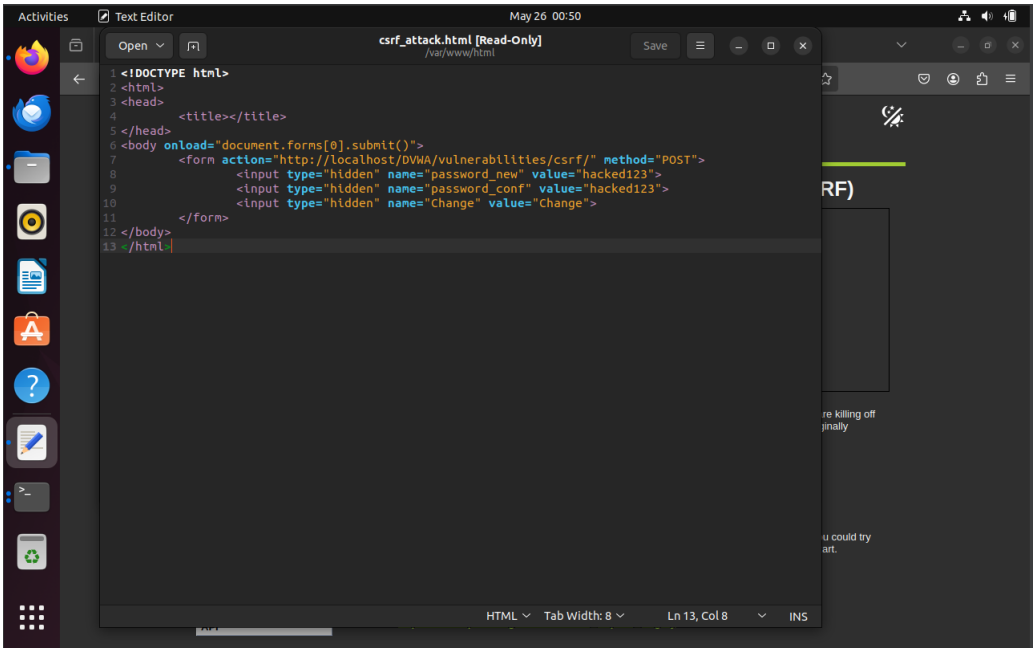
Summary: After injecting a script in the search box, an alert box popup in the website. This happens because the search box input isn't sanitized.

Reflection: Why is output encoding important?

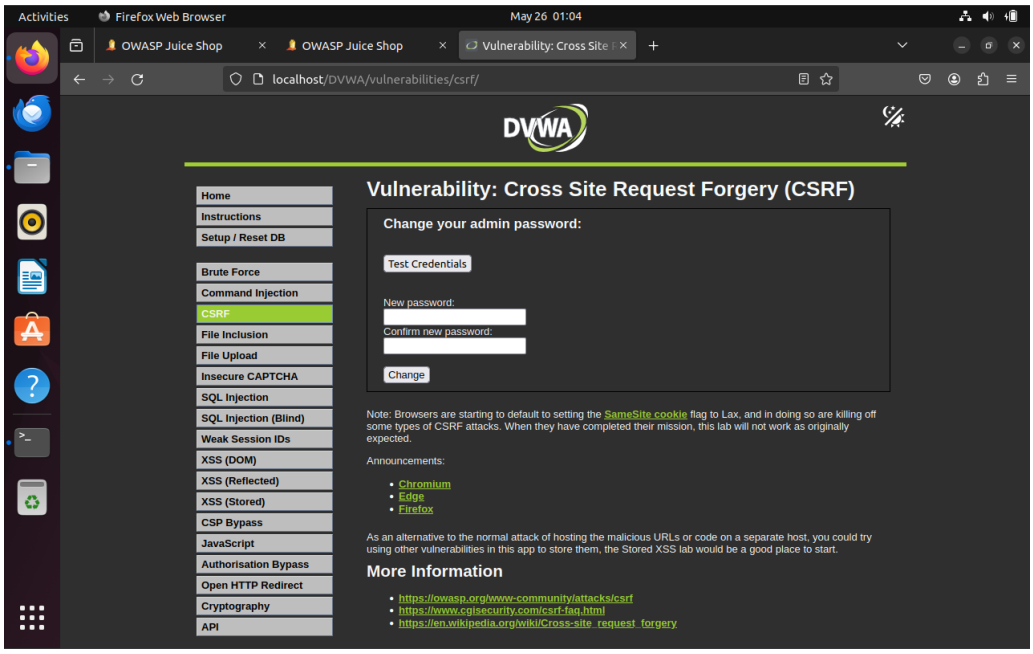
Answer: Output encoding is important because it helps prevent Cross-Site Scripting (XSS) and other injection attacks by ensuring that user-generated content is safely displayed in a web page without being interpreted as code.

4. CSRF (Cross-Site Request Forgery)

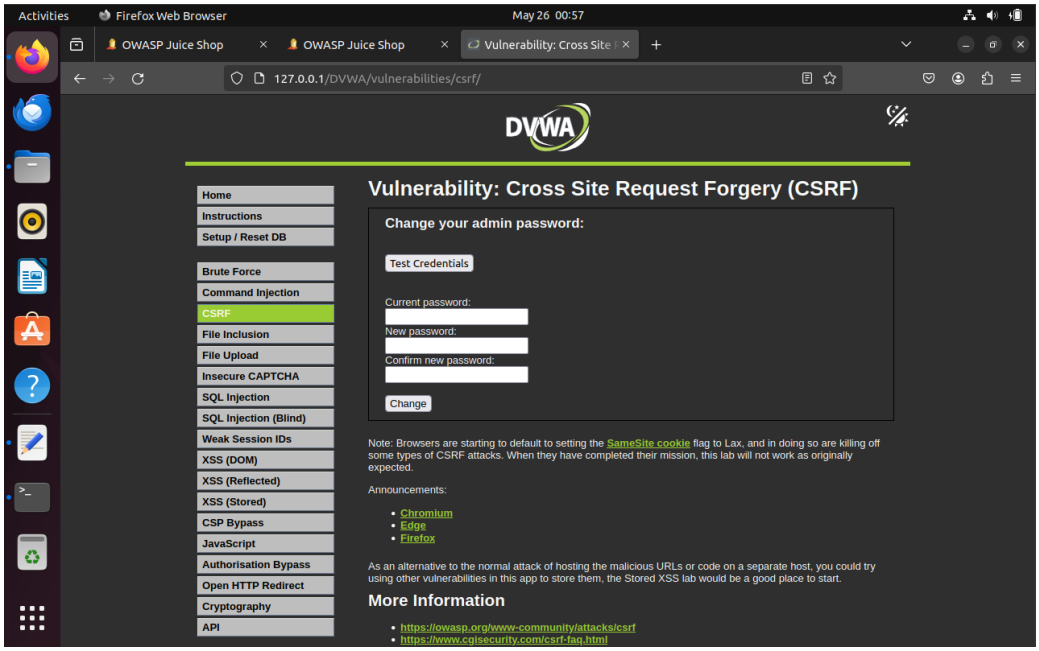
- Creating the HTML file



- Before forging the form



- After forging the form



Summary: Before forging the form, I was able to change my desired password. But after the forging the form and changing my password again, I wasn't able to login using my changed password (during active session) because the form in changing the password is forged.

Discussion: How does using CSRF tokens stop this attack?

Answer: The server embeds a hidden token in the form. When the form is submitted, the token is taken back. Then, the server checks if the token is present, if the token match what is issued, and if the token bound to the correct user. If all conditions are yes, then allows the request, else, it is rejected as a CSRF attempt.

5. Privilege Escalation

- Low-privileged user

```
user@debian:~$ whoami
user
user@debian:~$ id
uid=1000(user) gid=1000(user) groups=1000(user),24(cdrom),25(floppy),29(audio),3
0(dip),44(video),46(plugdev)
user@debian:~$ hostname
debian
user@debian:~$
```

- Checking sudo permissions and using ‘find’ to get to root access

```
user@debian:~$ sudo -l
Matching Defaults entries for user on this host:
    env_reset, env_keep+=LD_PRELOAD, env_keep+=LD_LIBRARY_PATH

User user may run the following commands on this host:
    (root) NOPASSWD: /usr/sbin/iftop
    (root) NOPASSWD: /usr/bin/find
    (root) NOPASSWD: /usr/bin/nano
    (root) NOPASSWD: /usr/bin/vim
    (root) NOPASSWD: /usr/bin/man
    (root) NOPASSWD: /usr/bin/awk
    (root) NOPASSWD: /usr/bin/less
    (root) NOPASSWD: /usr/bin/ftp
    (root) NOPASSWD: /usr/bin/nmap
    (root) NOPASSWD: /usr/sbin/apache2
    (root) NOPASSWD: /bin/more
user@debian:~$ sudo find . -exec /bin/sh \;
```

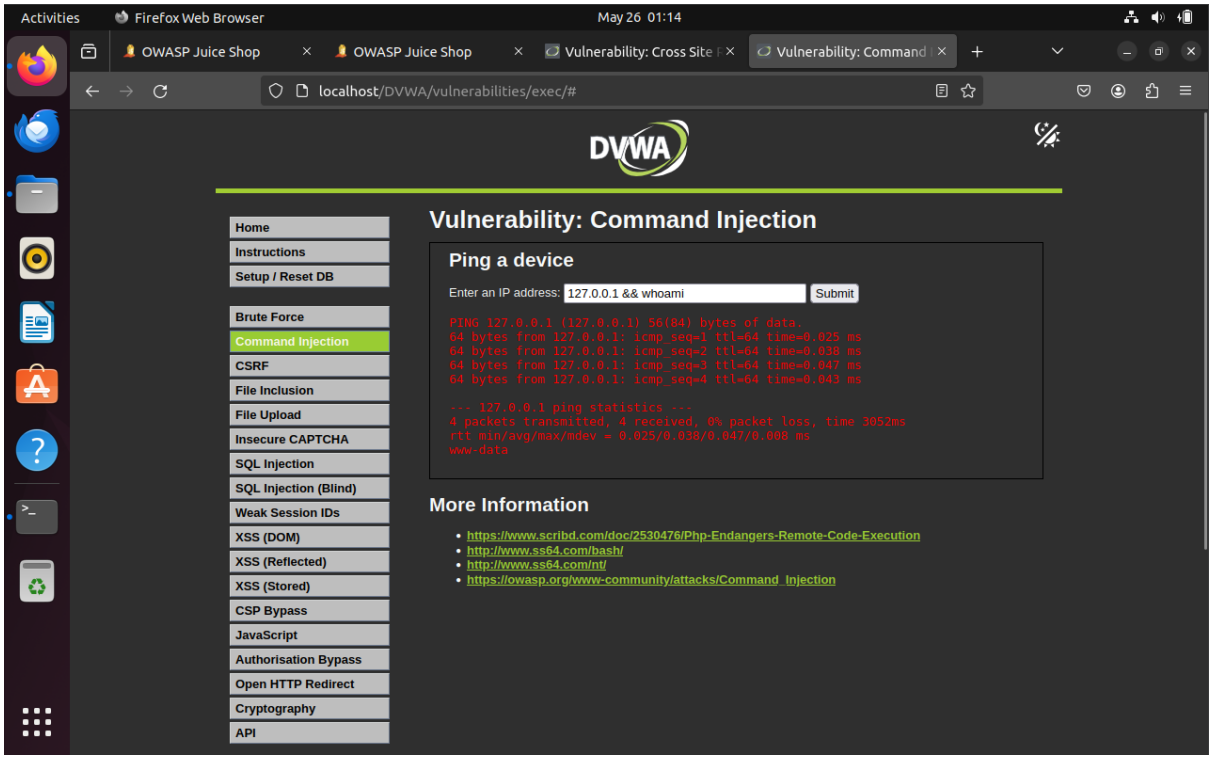
- Root Access

```
sh-4.1# whoami
root
sh-4.1# id
uid=0(root) gid=0(root) groups=0(root)
sh-4.1# hostname
debian
sh-4.1#
```

Summary: I was able to access root through exploiting a flaw in system or misconfigurations.

6. Remote Code Execution (RCE)

- Executing remote commands through user input.



Summary: I was able to ping a random ip address and get the current user of that ping using 'PING && whoami'. This happens because the ping input in DVWA vulnerable to special characters that may potentially use to run commands.

Defense: How can input sanitization stop this?

Answer: Input sanitization cleans, preventing or rejecting inputs that contain dangerous characters or patterns.

7. Denial of Service (DoS)

- Executing ping -f

```
sh-4.1# ping -f
Usage: ping [-LRUbdfnqrvVaAD] [-c count] [-i interval] [-w deadline]
            [-p pattern] [-s packetsize] [-t ttl] [-I interface]
            [-M pmtudisc-hint] [-m mark] [-S sndbuf]
            [-T tstamp-options] [-Q tos] [hop1 ...] destination
sh-4.1# sudo ping -f 10.10.90.213
PING 10.10.90.213 (10.10.90.213) 56(84) bytes of data.
.
```

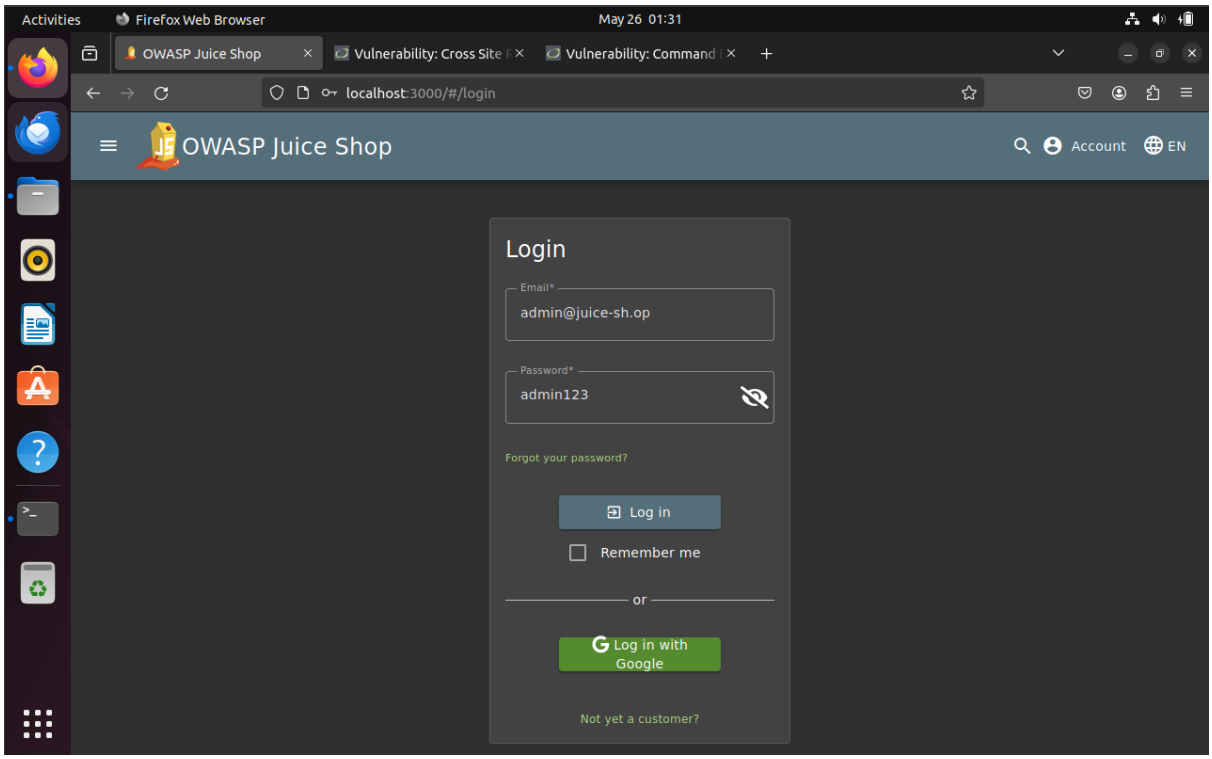
Summary: Since ping -f is really dangerous, I just ping own ip address (provided by TryHackMe) for testing purposes. After pinging myself, a rapid stream of dots appeared on my terminal sending a lot of packets. This DoS attack floods a system or server with excessive requests or data, overloading it so legitimate users can't access it. It causes temporary or permanent service outages.

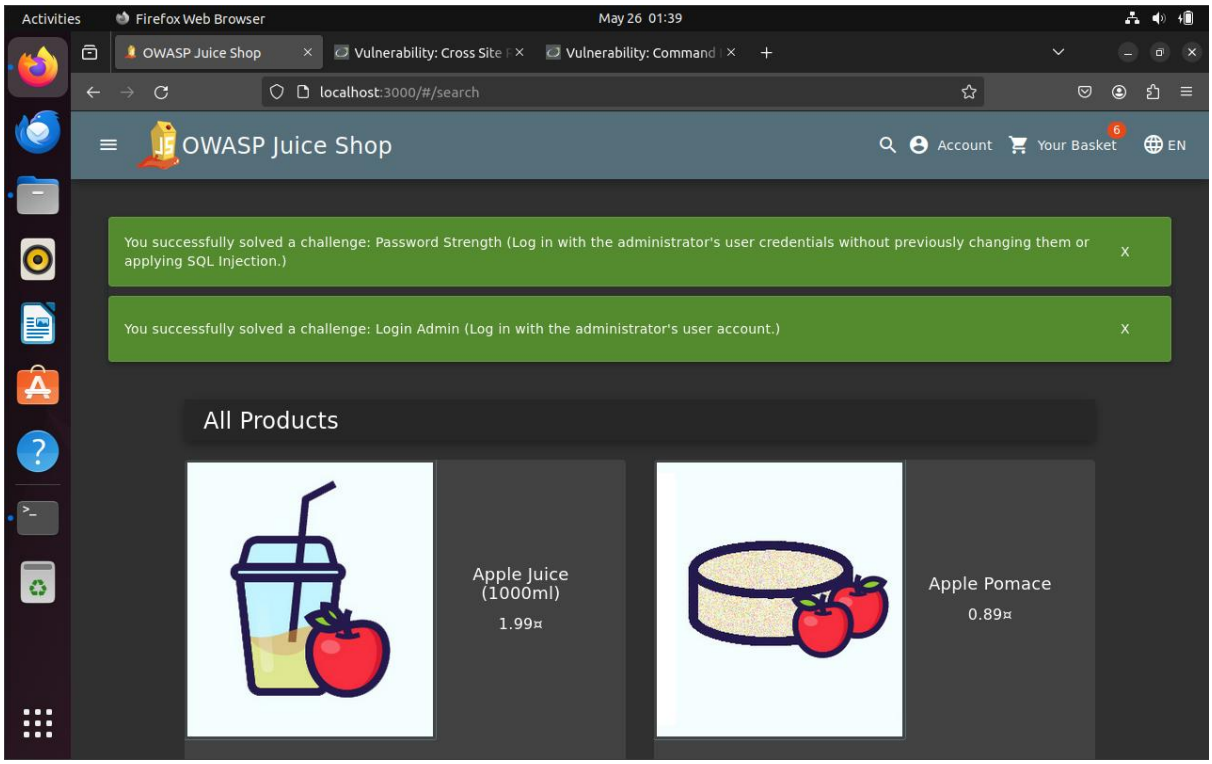
Discussion: What are rate limiting and firewalls' roles in defense?

Answer: The role of rate limiting in defense is to control the number of requests or packets a user can send to a server within a given time and the role of firewalls in defense is to filter incoming and outgoing traffic based on predefined security rules.

8. Authentication Bypass

- Exploiting default credentials





Summary: Accessed the website through default admin credentials. This happens because I trick the system into granting me access without proper login with poorly implemented security checks.

Mitigation: What practices help secure authentication mechanisms?

Answers: Best practices to help secure authentication mechanisms are: never deploy website/apps with default credentials; force password change on first login; enforce strong passwords; rate limit login attempts; second-factor authentication; use password encryptions; use CAPTCHA; and avoiding logic flaws.